

Synchronous FIFO

C++ Verification

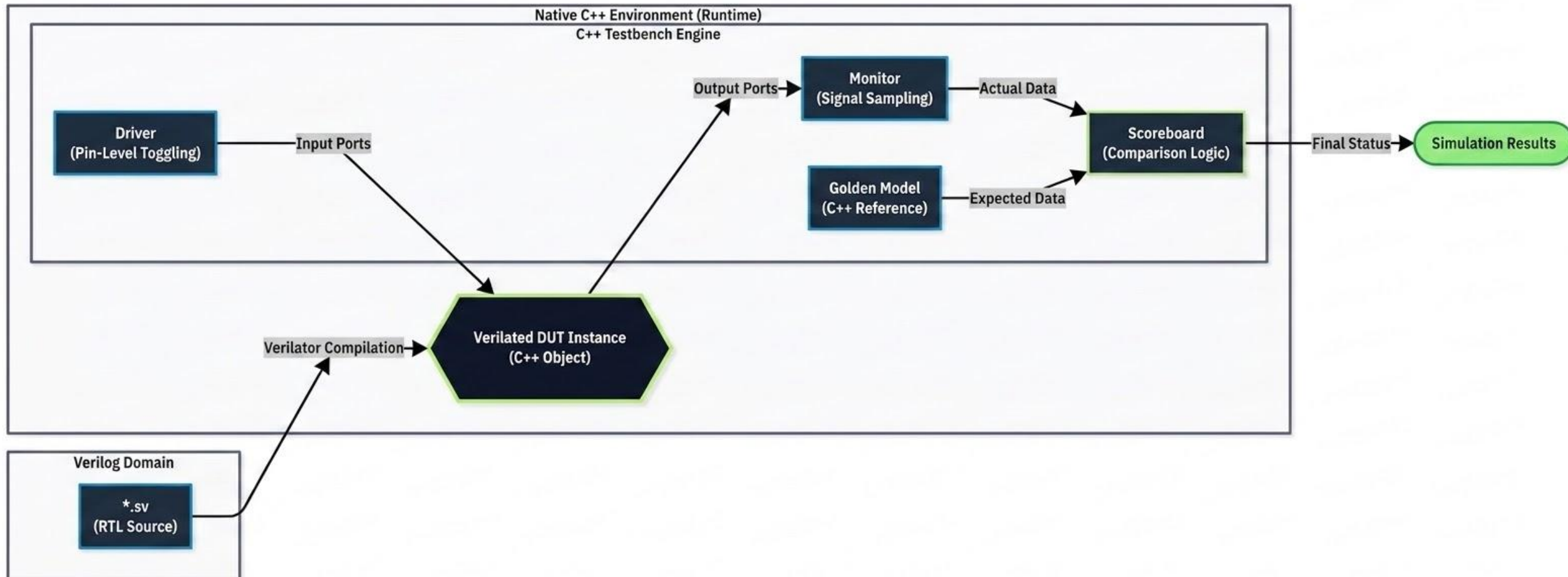
High-Performance Hardware Validation using Verilator & Native C++ Testbenches

GitHub:
[NguyenThanhTai0307/Synchronous-FIFO-CPP-Verification](https://github.com/NguyenThanhTai0307/Synchronous-FIFO-CPP-Verification)

Project Objectives

Optimizing the verification cycle by replacing traditional overhead-heavy UVM with a lightweight, high-performance C++ environment.

System Architecture



DUT: Synchronous FIFO

Design Specifications

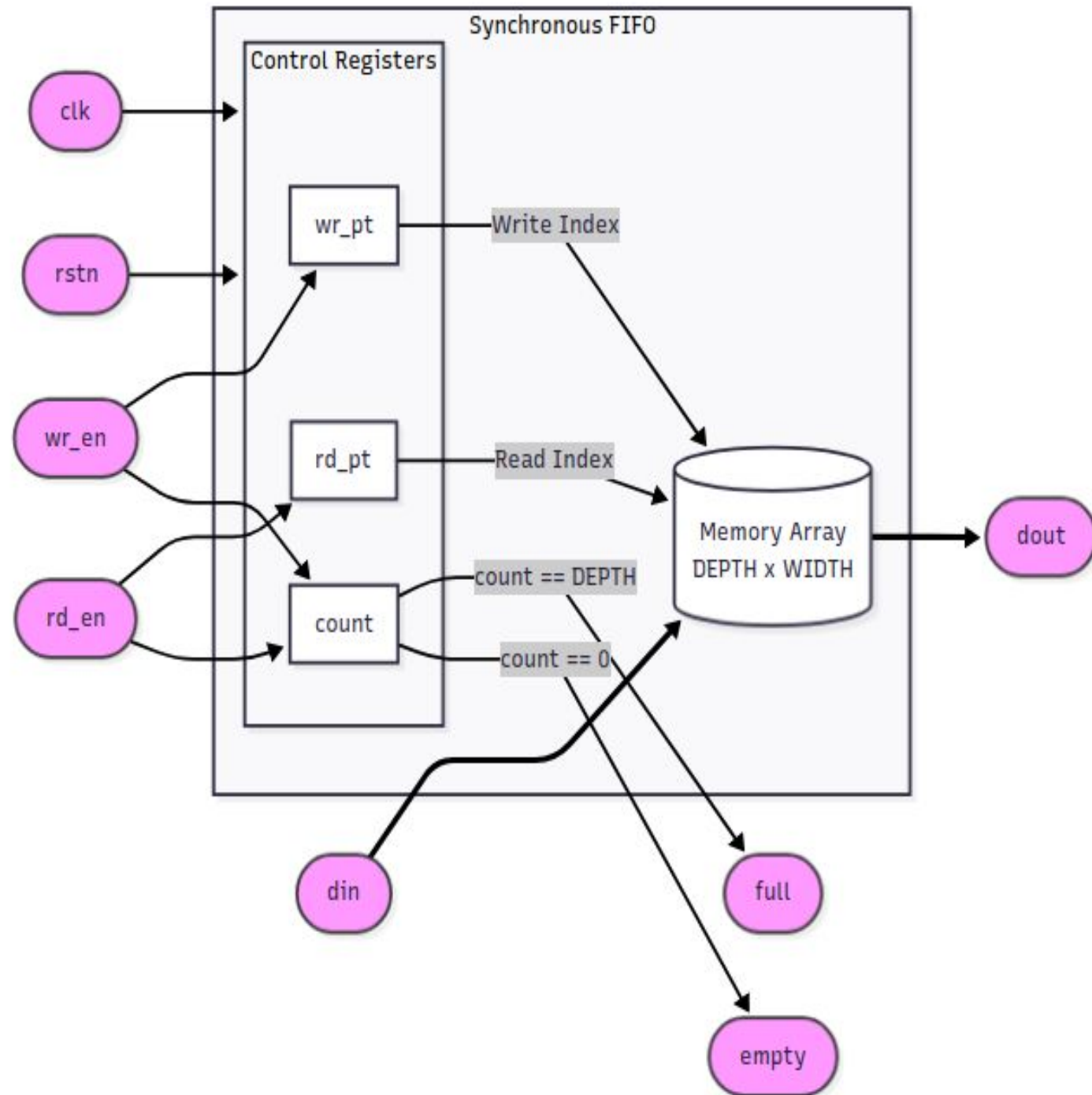
A high-performance buffer utilizing a dual-port RAM structure in a single clock domain.

Synchronous: Single clock for R/W.

Parameters: Configurable Depth $\times$ Width

Status: Full, Empty.

Safety: Overflow and Underflow protection.



| C++ Testbench Key Components



Driver

Translates transaction-level requests into pin-level signal toggles for the Verilated DUT.



Monitor

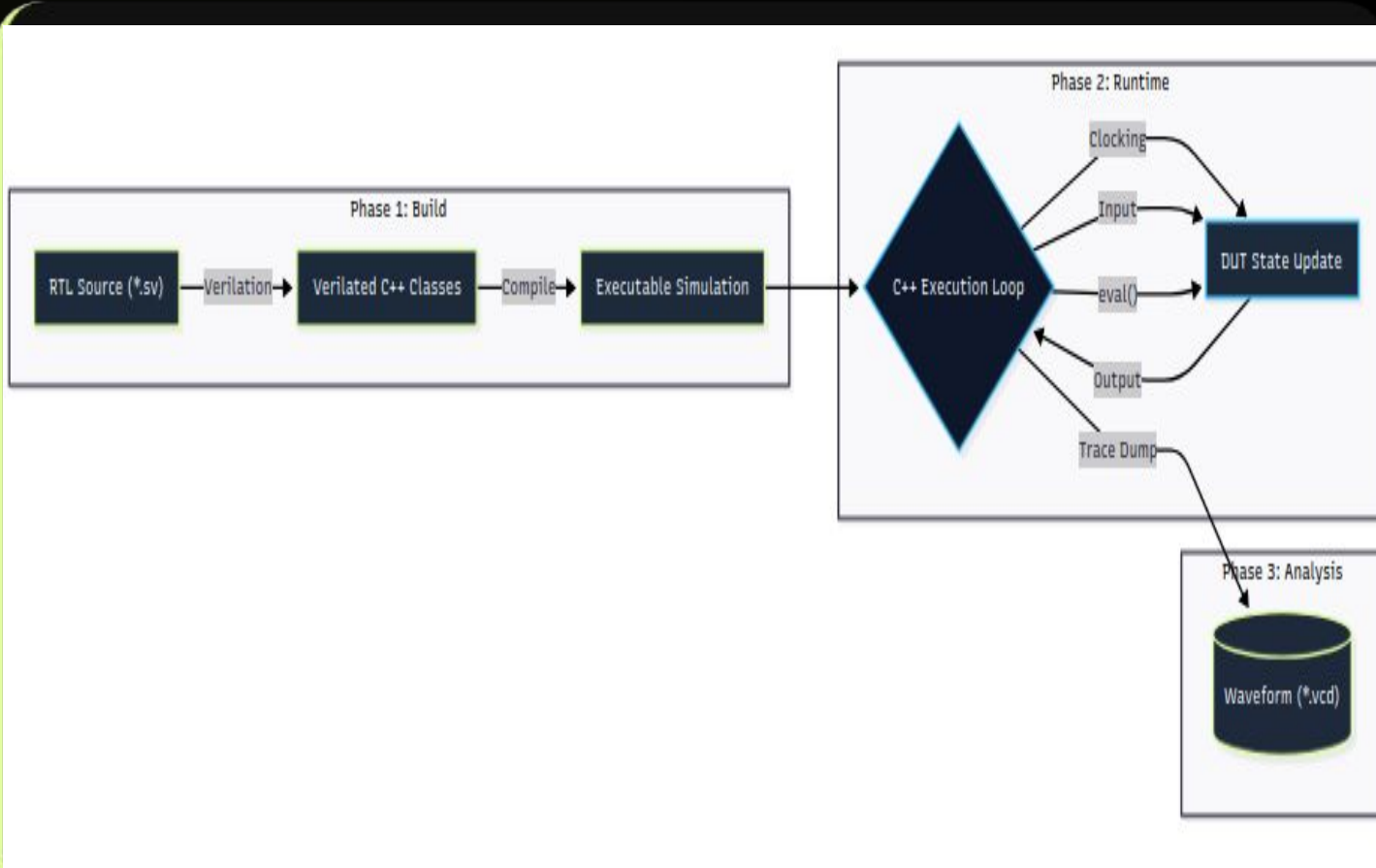
Passive observer that samples valid transactions from the interface and broadcasts to the scoreboard.



Scoreboard

Self-checking unit comparing DUT outputs with an internal C++ reference model (Golden Model).

Simulation Workflow



Key Processes

- ✓ **Verilation**: RTL to C++ compilation.
- ✓ **Clocking**: C++ owned time management.
- ✓ **VCD Trace**: Integrated .vcd generation.

| Verification Strategy



Constrained Random Testing: Using C++ standard libraries and custom constraints to generate diverse input vectors.



Directed Boundary Testing: Forcing "Full" and "Empty" states to verify transition logic and flag accuracy.



Corner Case Injection: Testing simultaneous Read and Write operations at capacity boundaries.

Data Consistency: Continuous scoreboard checking to ensure data order and integrity are preserved.

Test Matrix Results

Test Case ID	Verification Objective	Status
FIFO_RESET_TEST	Verify initial flag states and pointer reset logic	PASSED
FIFO_OVERFLOW_TEST	Attempt write to Full FIFO / read from Empty FIFO	PASSED
FIFO_SIMUL_RW_TEST	Validate data integrity during concurrent R/W cycles	PASSED
RANDOM_DATA_TEST	5000+ iterations of randomized burst R/W cycles	PASSED

Waveform & Results

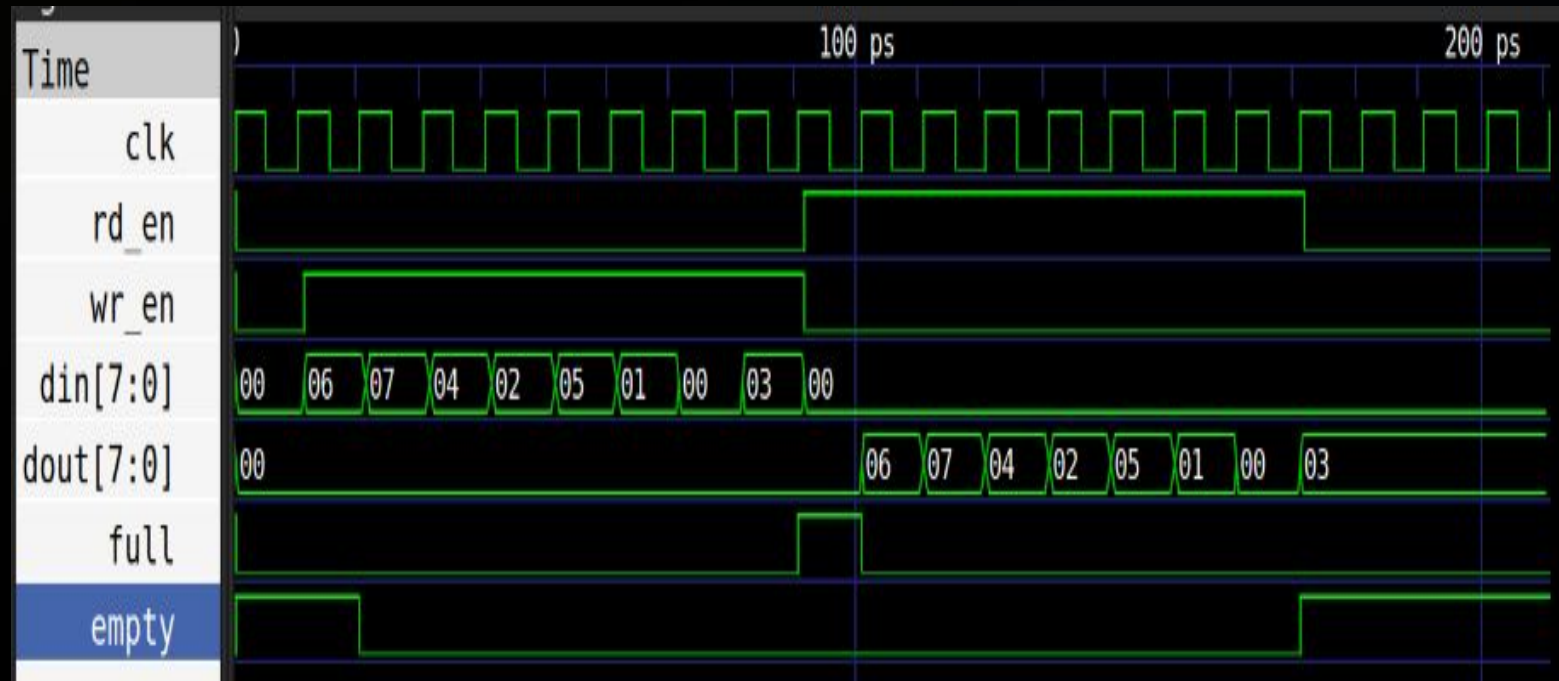
GTKWave Insights

Simulation traces confirm perfect alignment of wr_en with full assertion and rd_en with empty de-assertion.

 Sub-nanosecond signal resolution.

 Accurate status flag tracking.

 Efficient memory array tracing.





Questions?

Thank you for your attention.

Repository: github.com/NguyenThanhTai0307

Modern Hardware Verification with Verilator & C++